Indian Voice Agent Platform: Complete Implementation Guide

Version: 1.0

Target Stack: FastAPI (Backend), Next.js + React Flow (Frontend), Sarvam AI (Models), Firebase + Cloud Run (Deployment)

Table of Contents

1. Executive Summary & Architecture
2. Chapter 1: The "Indian" Model Layer (Sarvam Integration)
3. Chapter 2: The Core Engine (FastAPI + Twilio)
4. Chapter 3: The Workflow Engine (Logic & State)
5. Chapter 4: The Frontend Builder (React Flow)
6. Chapter 5: Post-Call Analytics (Oration Style)
7. Chapter 6: Deployment Strategy (Firebase & Cloud Run)

1. Executive Summary & Architecture

This platform enables users to visually design voice agents (ElevenLabs style) that are specifically optimized for Indian languages (Hindi, Hinglish, Tamil, etc.) using Sarvam AI's models.

High-Level Data Flow

1. **Phone Call:** Customer calls a Twilio number.
2. **WebSocket:** Twilio connects to our **FastAPI** backend via WebSocket.
3. **STT (Ear):** Incoming audio (Mulaw) is transcoded and sent to **Sarvam Saaras** (via WebSocket) for real-time transcription.
4. **Brain:** The **Workflow Engine** processes the text against the user-defined JSON graph.
5. **TTS (Mouth):** Response text is sent to **Sarvam Bulbul V3**.
6. **Output:** Audio is played back to the caller.
7. **Analysis:** After the call, the transcript is sent to an LLM for "Oration-style" extraction (Sentiment, Intent, Action Items).

Tech Stack

- **Backend:** Python 3.11, FastAPI, Uvicorn, Websockets.
- **Frontend:** Next.js 14, React Flow, TailwindCSS, Shadcn UI.
- **Database:** Firestore (for storing agent configurations and call logs).
- **Telephony:** Twilio (Media Streams).

- **AI Models:**
  - TTS: Sarvam Bulbul V3 (Hindi/English).
  - STT: Sarvam Saaras (Streaming).
  - Analytics: Gemini Flash / GPT-4o-mini.

2. Chapter 1: The "Indian" Model Layer

To build a robust system, we must not hardcode Sarvam. We use the **Provider Pattern**. This allows us to swap Sarvam for Deepgram or OpenAI later if needed, but defaults to Sarvam for now.

2.1 Abstract Base Classes

Create a file app/services/ai_providers.py:

```
from abc import ABC, abstractmethod
from typing import AsyncGenerator, Optional

class STTProvider(ABC):
    """Abstract Speech-to-Text Provider"""
    @abstractmethod
    async def connect(self):
        """Establish connection to the provider"""
        pass

    @abstractmethod
    async def send_audio(self, audio_chunk: bytes):
        """Send raw audio bytes"""
        pass

    @abstractmethod
    async def receive_text(self) -> AsyncGenerator[str, None]:
        """Yield transcribed text"""
        pass

class TTSProvider(ABC):
    """Abstract Text-to-Speech Provider"""
    @abstractmethod
    async def generate_audio(self, text: str, voice_id: str, language: str) -> bytes:
        """Return audio bytes for the given text"""
        pass
```

2.2 Sarvam TTS Implementation (Bulbul V3)

Sarvam's TTS is best accessed via REST for stability, though streaming is available. We will use the synchronous REST API wrapped in async for simplicity in the first version.

```
import aiohttp
import base64
import os

class SarvamTTS(TTSProvider):
    def __init__(self):
        self.api_key = os.getenv("SARVAM_API_KEY")
        self.url = "[https://api.sarvam.ai/text-to-speech](https://api.sarvam.ai/text-to-speech)"

    async def generate_audio(self, text: str, voice_id: str = "bulbul:v3", language: str = "hi-IN") ->
bytes:
        payload = {
            "inputs": [text],
            "target_language_code": language,
            "speaker": "meera", # Example speaker
            "pitch": 0,
            "pace": 1.0,
            "loudness": 1.5,
            "speech_sample_rate": 8000, # Crucial for Telephony (Twilio)
            "enable_preprocessing": True,
            "model": "bulbul:v3"
        }

        headers = {"Ocp-Apim-Subscription-Key": self.api_key, "Content-Type":
"application/json"}

        async with aiohttp.ClientSession() as session:
            async with session.post(self.url, json=payload, headers=headers) as response:
                if response.status != 200:
                    error = await response.text()
                    raise Exception(f"Sarvam TTS Error: {error}")

                data = await response.json()
                # Sarvam returns base64 audio
                audio_base64 = data["audios"][0]
                return base64.b64decode(audio_base64)
```

2.3 Sarvam STT Implementation (Saaras Streaming)

This is the hardest part. Twilio sends 8kHz Mulaw audio. Sarvam expects PCM. We must transcode.

```
import websockets
import json
import audioop

class SarvamSTT(STTProvider):
    def __init__(self):
        self.api_key = os.getenv("SARVAM_API_KEY")
        self.ws_url = "wss://api.sarvam.ai/speech-to-text-streaming"
        self.websocket = None

    async def connect(self):
        # We assume the URL structure based on their streaming docs
        # Note: In production, check for exact query params like ?model=saaras:v1
        self.websocket = await websockets.connect(
            f"{self.ws_url}?model=saaras:v1",
            extra_headers={"Ocp-Apim-Subscription-Key": self.api_key}
        )

    async def send_audio(self, audio_chunk_mulaw: bytes):
        if not self.websocket:
            return

        # 1. Transcode Twilio Mulaw -> PCM 16-bit
        # Twilio sends 8kHz mulaw.
        pcm_data = audioop.ulaw2lin(audio_chunk_mulaw, 2)

        # 2. Sarvam usually expects raw bytes or a JSON frame depending on version
        # Assuming raw bytes for this example, or a JSON wrapper:
        await self.websocket.send(pcm_data)

    async def receive_text(self):
        async for message in self.websocket:
            data = json.loads(message)
            # Sarvam streaming response structure (example)
            if data.get("type") == "transcript" and data.get("transcript"):
                yield data["transcript"]
```

3. Chapter 2: The Core Engine (FastAPI + Twilio)

The backend creates a persistent WebSocket connection for the duration of the call.

3.1 main.py Setup

```
from fastapi import FastAPI, WebSocket, Request
from fastapi.responses import HTMLResponse
from app.core.connection_manager import ConnectionManager

app = FastAPI()
manager = ConnectionManager()

@app.post("/voice")
async def voice_webhook(request: Request):
    """
    Twilio hits this URL when a call starts.
    We return TwiML to tell Twilio to connect to our WebSocket.
    """
    host = request.url.hostname
    return HTMLResponse(content=f"""
    <Response>
        <Connect>
            <Stream url="wss://{host}/ws/call" />
        </Connect>
    </Response>
    """, media_type="application/xml")

@app.websocket("/ws/call")
async def websocket_endpoint(websocket: WebSocket):
    await manager.handle_call(websocket)
```

3.2 The Connection Manager (Orchestrator)

This class handles the messy details of the Twilio Media Stream protocol.

```
# app/core/connection_manager.py
import json
import base64
import asyncio
```

```
from app.services.ai_providers import SarvamTTS, SarvamSTT
from app.core.workflow_engine import WorkflowEngine

class ConnectionManager:
    async def handle_call(self, websocket: WebSocket):
        await websocket.accept()

        # Initialize AI
        stt = SarvamSTT()
        tts = SarvamTTS()

        # Initialize Workflow (Hardcoded for now, dynamic later)
        workflow_engine = WorkflowEngine(agent_id="agent_1", tts=tts)

        await stt.connect()

        stream_sid = None

        try:
            async for message in websocket.iter_text():
                data = json.loads(message)
                event = data['event']

                if event == "start":
                    stream_sid = data['start']['streamSid']
                    # Trigger the "Start" node of the workflow
                    intro_audio = await workflow_engine.start()
                    if intro_audio:
                        await self.send_audio(websocket, stream_sid, intro_audio)

                elif event == "media":
                    # 1. Get Audio from Twilio
                    payload = data['media']['payload']
                    chunk = base64.b64decode(payload)

                    # 2. Send to STT
                    await stt.send_audio(chunk)

                    # 3. Check for Transcripts (Non-blocking usually done in separate task)
                    # For simplicity, we assume stt.receive_text() is handled in a background task

        except Exception as e:
            print(f"Call Error: {e}")
```

```
        finally:
            await websocket.close()

    async def send_audio(self, ws, stream_sid, audio_bytes):
        # Convert PCM/WAV back to Mulaw for Twilio
        # audio_mulaw = audioop.lin2ulaw(audio_bytes, 2)
        # Note: Sarvam might return WAV. You need to strip headers and convert.

        payload = base64.b64encode(audio_bytes).decode('utf-8')
        message = {
            "event": "media",
            "streamSid": stream_sid,
            "media": {"payload": payload}
        }
        await ws.send_text(json.dumps(message))
```

4. Chapter 3: The Workflow Engine

This is the "Brain" that mimics ElevenLabs/Oration agents. It follows a JSON graph.

3.1 The JSON Schema

```
{
  "id": "agent_healthcare_1",
  "nodes": [
    {
      "id": "start",
      "type": "speak",
      "text": "Namaste! This is Dr. Batra's clinic. Are you calling for an appointment?",
      "next": "listen_intent"
    },
    {
      "id": "listen_intent",
      "type": "listen",
      "options": [
        {"intent": "yes", "next": "ask_time"},
        {"intent": "no", "next": "end_call"}
      ]
    },
    {
      "id": "ask_time",
      "type": "speak",
```

```
      "text": "Great. What time works for you?",
      "next": "listen_time"
    }
  ]
}
```

3.2 The Engine Logic

```
# app/core/workflow_engine.py

class WorkflowEngine:
    def __init__(self, agent_id, tts):
        self.graph = self.load_graph(agent_id) # Loads JSON from DB
        self.current_node_id = "start"
        self.tts = tts

    async def start(self):
        return await self.process_node(self.current_node_id)

    async def process_node(self, node_id):
        node = self.find_node(node_id)

        if node['type'] == 'speak':
            # Generate Audio
            audio = await self.tts.generate_audio(node['text'])

            # Move pointer immediately or wait?
            # In a real app, we wait for 'media completion' event from Twilio
            self.current_node_id = node['next']
            return audio

        elif node['type'] == 'listen':
            # This state waits for STT input
            return None

    async def handle_user_input(self, text):
        # Called when STT gives us text
        current_node = self.find_node(self.current_node_id)

        if current_node['type'] == 'listen':
            # Simple keyword matching or LLM router
            intent = await self.classify_intent(text, current_node['options'])
```

```
            next_node_id = intent['next']
            self.current_node_id = next_node_id
            return await self.process_node(next_node_id)
```

5. Chapter 4: The Frontend Builder

We use **Next.js** and **React Flow**.

5.1 Directory Structure

```
/frontend
  /components
    /builder
      Canvas.tsx       # The main React Flow area
      Sidebar.tsx      # Drag items (Speak, Listen, API)
      /nodes
        SpeakNode.tsx   # Custom Node UI
        ListenNode.tsx
  /app
    /dashboard
      page.tsx         # List of Agents
    /builder/[id]
      page.tsx         # The Editor
```

5.2 Custom Node: SpeakNode.tsx

This mimics the ElevenLabs node design.

```
import { Handle, Position } from 'reactflow';

export default function SpeakNode({ data }) {
  return (
    <div className="bg-white border-2 border-blue-500 rounded-lg p-4 shadow-md w-64">
      <div className="font-bold mb-2 text-gray-700">🤖 Agent Says</div>
      <textarea
        className="w-full text-sm border p-2 rounded"
        placeholder="Enter text (e.g., Namaste...)"
        defaultValue={data.text}
        onChange={(e) => data.onChange(e.target.value)}
      />
      <div className="text-xs text-gray-400 mt-2">Voice: Sarvam / Meera</div>
```

```
      <Handle type="source" position={Position.Right} className="bg-blue-500" />
      <Handle type="target" position={Position.Left} className="bg-blue-500" />
    </div>
  );
}
```

5.3 Saving the Flow

When the user clicks "Save", we traverse the React Flow nodes and edges arrays and convert them into our Engine's JSON format.

```
const onSave = () => {
  const engineFormat = {
    id: agentId,
    nodes: reactFlowNodes.map(node => ({
      id: node.id,
      type: node.type,
      text: node.data.text,
      next: edges.find(e => e.source === node.id)?.target
    }))
  };
  // Save to Firestore
  saveToFirebase(engineFormat);
}
```

6. Chapter 5: Post-Call Analytics (Oration Style)

Oration AI provides detailed tags and scores. We do this via a background worker.

6.1 The Logic

1. **FastAPI Background Task:** When the websocket closes (finally block), trigger this function.
2. **Input:** The transcript array [{role: 'agent', text: '...'}, {role: 'user', text: '...'}].
3. **LLM Call:**

```
async def analyze_call(transcript):
    prompt = f"""
    Analyze this sales call transcript.
    Return JSON:
```

```
{{
   "sentiment": "Positive" | "Neutral" | "Negative",
   "success": boolean,
   "summary": "string",
   "action_items": ["item1", "item2"]
}}
Transcript: {transcript}
"""
# Call Gemini Flash or GPT-4o
result = await call_llm(prompt)

# Save to Firestore
db.collection('calls').add(result)
```

7. Chapter 6: Deployment Strategy

Since you specifically asked about Firebase deployment:

7.1 Frontend (Firebase Hosting)

1. Build the Next.js app: npm run build.
2. Configure firebase.json:
   ```
   {
     "hosting": {
       "source": ".",
       "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
       "frameworksBackend": {
         "region": "asia-south1"
       }
     }
   }
   ```
3. Deploy: firebase deploy --only hosting.

7.2 Backend (Google Cloud Run)

Standard Firebase Functions (Gen 2) have a timeout of 60 minutes, which is *okay* for calls, but **Cloud Run** is safer for WebSockets and cheaper for high concurrency.

1. **Dockerfile:**
   ```
   FROM python:3.11-slim
   WORKDIR /app
   COPY requirements.txt .
   ```

```
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

2. **Build & Deploy:**
```
gcloud run deploy voice-agent-backend \
  --source . \
  --region asia-south1 \
  --allow-unauthenticated
```

3. **Link to Firebase:** You can treat this Cloud Run service as your API backend. In your Next.js app, point your API calls to the Cloud Run URL.

7.3 Twilio Configuration

Update your Twilio Phone Number webhook to point to your Cloud Run URL:

https://voice-agent-backend-xyz.a.run.app/voice

Conclusion

This architecture gives you the flexibility of **ElevenLabs** (Graph UI), the intelligence of **Oration AI** (Post-call analytics), and the localization of **Sarvam AI** (Indian languages). By using the Abstract Provider pattern, you future-proof the application against API changes.